

Module 2 – Introduction to Programming

Q - 1 : Write an essay covering the history and evolution of C programming. Explain its importance and why it is still used today.

A - 1 : C was developed by **Dennis Ritchie** at Bell Labs in the early 1970s. It evolved from the B language and was used to rewrite the Unix operating system.

Importance: It is a "middle-level" language, combining the speed of low-level assembly with the ease of high-level languages.

Today: It remains vital because it is fast, portable, and allows direct memory access, making it the backbone of modern computing.

Q - 2 : Describe the steps to install a C compiler (e.g., GCC) and set up an Integrated Development Environment (IDE) like DevC++, VS Code, or CodeBlocks.

A - 2 : Steps to Install C Compiler and IDE

1. Download and install GCC compiler (MinGW/TDM-GCC).
2. Add the compiler bin path to Environment Variables.
3. Verify installation using:

gcc --version

Dev-C++

- Install Dev-C++.
- Open → Create new source file → Write code → Compile & Run.

VS Code

- Install VS Code and C/C++ extension.
- Open terminal and compile using:

gcc file.c -o file

CodeBlocks

- Install CodeBlocks with MinGW.
- Create Console Application → Write code → Build & Run.

Q - 3 : Explain the basic structure of a C program, including headers, main function, comments, data types, and variables. Provide examples.

A - 3 : C program consists of different parts such as headers, main function, comments, data types, and variables.

Key Components :

- **Header:** #include <stdio.h> (Input/Output library).
- **Main Function:** int main() is the entry point where execution starts.
- **Variables:** Containers for data (e.g., int age = 18;).
- **Comments:** // for single line, /* */ for multi-line.

Q - 4 : Write notes explaining each type of operator in C: arithmetic, relational, logical, assignment, increment/decrement, bitwise, and conditional operators.

A - 4 : · **Arithmetic:** +, -, *, /, % (Math)

· **Relational:** ==, !=, >, < (Comparison)

· **Logical:** && (AND), || (OR), ! (NOT)

· **Assignment:** = (Store value)

Q - 5 : Explain decision-making statements in C (if, else, nested if-else, switch). Provide examples of each.

A - 5 : Decision-making statements are used to control the flow of a program based on conditions.

1. if Statement

The if statement executes a block of code only if the condition is true.

```
#include<stdio.h>

int main()
{
```

```
int age = 18;

if(age >= 18)
{
    printf("You are eligible to vote");
}
return 0;
}
```

2. if-else Statement

The if-else statement executes one block if the condition is true and another block if it is false.

```
#include<stdio.h>

int main()
{
    int num = 5;

    if(num % 2 == 0)
    {
        printf("Even Number");
    }
    else
    {
        printf("Odd Number");
    }
    return 0;
}
```

3. Nested if-else Statement

A nested if-else means an if statement inside another if or else.

```
#include<stdio.h>

int main()
{
int num = 10;

if(num > 0)
{
if(num % 2 == 0)
{
printf("Positive Even Number");
}
else
{
printf("Positive Odd Number");
}
}
else
{
printf("Negative Number");
}

return 0;
}
```

4. switch Statement

The switch statement is used to select one option from many choices.

```
#include<stdio.h>

int main()
{
    int day = 2;

    switch(day)
    {
        case 1:
            printf("Monday");
    }
}
```

```

        break;

    case 2:
        printf("Tuesday");
        break;

    case 3:
        printf("Wednesday");
        break;

    default:
        printf("Invalid Day");
    }

    return 0;
}

```

Q - 6 : Compare and contrast while loops, for loops, and do-while loops. Explain the scenarios in which each loop is most appropriate.

A - 6 : · **for:** Best when you know exactly how many times to repeat.

- **while:** Best when the loop depends on a condition being met.
- **do-while:** Guaranteed to run at least once.

Q - 7 : Explain the use of break, continue, and goto statements in C. Provide examples of each

A - 7 : · **break:** Exits the loop entirely.

- **continue:** Skips the current iteration and goes to the next.
- **goto:** Jumps to a specific labeled part of the program (rarely used).

Q - 8 : What are functions in C? Explain function declaration, definition, and how to call a function. Provide examples

A - 8 : A function in C is a block of code that performs a specific task. Functions help make programs modular, reusable, and easier to understand

1. Function Declaration : A function declaration tells the compiler about the function name, return type, and parameters before it is used.

2. Function Definition : A function definition contains the actual code of the function.

3. Function Call : A function call is used to execute the function.

```
#include<stdio.h>

// Function Declaration
int add(int, int);

int main()
{
    int result;

    // Function Call
    result = add(5, 3);

    printf("Sum = %d", result);

    return 0;
}

// Function Definition
int add(int a, int b)
{
    return a + b;
}
```

Q - 9 : Explain the concept of arrays in C. Differentiate between one-dimensional and multi-dimensional arrays with examples.

- 1D Array: A single list of items (e.g., int marks[5]).
- 2D Array: A table with rows and columns (e.g., int matrix[3][3]).

Q - 10 : Explain what pointers are in C and how they are declared and initialized. Why are pointers important in C?

A - 10 : A pointer in C is a variable that stores the memory address of another variable.

- Pointers are one of the most powerful features of the C language.
- A pointer is declared using the `*` symbol.

Important of Pointers in C :

1. Dynamic Memory Allocation: Pointers are used with functions like `malloc()` and `calloc()` to allocate memory dynamically.

2. Efficient Function Calls: Pointers allow functions to modify original variables using call by reference.

3. Array and String Handling: Arrays and strings are accessed using pointers.

4. Data Structures: Pointers are essential for linked lists, trees, stacks, and queues.

5. Memory Management: Pointers provide direct access to memory locations.

Q - 11 : Explain string handling functions like `strlen()`, `strcpy()`, `strcat()`, `strcmp()`, and `strchr()`. Provide examples of when these functions are useful.

A - 11 : String handling functions are predefined functions available in the header file `string.h`. They are used to perform operations on strings.

1. `strlen()` – String Length : The `strlen()` function returns the length of a string (excluding `\0`).

2. `strcpy()` – Copy String : The `strcpy()` function copies one string into another.

3. `strcat()` – Concatenate Strings : The `strcat()` function joins two strings.

4. `strcmp()` – Compare Strings : The `strcmp()` function compares two strings.

5. strchr() – Search Character in String : The strchr() function finds the first occurrence of a character in a string.

Q - 12 : Explain the concept of structures in C. Describe how to declare, initialize, and access structure members

A - 12 : A structure in C is a user-defined data type that allows storing different types of data together under one name.

- Structures are useful when we need to group related data.

Declaration of Structure : A structure is declared using the struct keyword.

Q - 13 : Explain the importance of file handling in C. Discuss how to perform file operations like opening, closing, reading, and writing files.

A - 13 : File handling in C is used to store data permanently in files instead of keeping it only in memory. It allows programs to create, read, write, and modify files.

- C provides file handling functions in the standard library stdio.h.

Importance of File Handling :

- 1. Permanent Data Storage :** Data remains saved even after the program ends.
- 2. Large Data Management :** Files can store large amounts of data efficiently.
- 3. Data Sharing :** Files help share data between programs.
- 4. Record Keeping :** Useful for student records, employee data, reports, etc.

File Operations in C :

- 1. Opening a File :** A file is opened using the fopen() function.

2. Writing to a File : The fprintf() function is used to write data into a file.

3. Reading from a File : The fscanf() function is used to read data from a file.

4. Closing a File : The fclose() function closes the file after operations are completed.